



HyperRAM Controller Core User Guide

UG-CORE-HYPERRAM-v4.1

December 2025

www.efinixinc.com



Contents

Introduction.....	3
Features.....	3
Device Support.....	3
Resource Utilization and Performance.....	4
Release Notes.....	4
Functional Description.....	5
Ports.....	8
AXI User Interface.....	14
AXI Addressing.....	14
Unaligned Address Access.....	15
AXI Read and Write Operation.....	15
Native User Interface.....	16
PLL Auto Calibration Flow.....	17
Periodic Recalibration Flow.....	19
On-Demand Calibration.....	20
PLL Manual Calibration Flow.....	21
HyperRAM Controller Operation.....	22
Dynamic Reconfiguration for Configuration Register.....	25
Configuration Register.....	28
Configuration Register 0.....	28
Configuration Register 1.....	29
Interface Designer Settings.....	30
PLL Auto Calibration and PLL Manual Calibration.....	30
IP Manager.....	33
Customizing the HyperRAM Controller.....	34
HyperRAM Controller Example Design.....	37
Virtual I/O Debugger Settings.....	39
HyperRAM Controller Testbench.....	39
Revision History.....	40

Introduction

HyperRAM is a memory device that uses the HyperBus protocol. The HyperRAM Controller has two width options, x8 (13 I/O pins) and x16 (22 I/O pins). This flexibility allows designers to reduce the number of traces needed on the printed circuit board and thus is ideal for scalable solutions especially in automotive, industrial, and IoT applications. The HyperRAM Controller core interfaces Titanium and Topaz FPGAs with HyperRAM memories.

Use the IP Manager to select IP, customize it, and generate files. The HyperRAM Controller core has an interactive wizard to help you set parameters. The wizard also has options to create a testbench and/or example design targeting an Efinix® development board.



Note: The HyperRAM Controller is only meant to communicate with single discrete HyperRAM chip.

Features

- PLL auto calibration and PLL manual calibration
- Various periodic calibration capabilities
- x8 and x16 RAM bit widths
- Dual HyperRAM support
- Supports double-data rates of up to 800 MBps for x16 width configuration (up to 1000 MBps for Ti60 F100S3F2/ Ti35 F100S3F2 non-L devices)
- Supports up to 512 Mb HyperRAM
- Linear and wrap burst transfer
- Dynamic reconfiguration of configuration transfer
- Hybrid sleep and deep power down support
- AXI3 half-duplex or native interface to core
- 32, 64, 128, and 256 bit data width
- Includes Verilog HDL RTL and simulation testbench
- Includes example designs targeting the Titanium Ti60 F225 Development Board and Titanium Ti60 F100S3F2 FPGA

Device Support

Table 1: HyperRAM Controller Core Device Support

FPGA Family	Supported Device
Titanium	All
Topaz	All

Efinix® have verified HyperRAM Controller core for the Winbond (part no: W958D6NWS) HyperRAM with Titanium FPGAs.

Resource Utilization and Performance



Note: The resources and performance values provided are based on some of the supported FPGAs. These values are just guidance and may change depending on the device resource utilization, design congestion, and user design.

Table 2: Titanium Resource Utilization and Performance

FPGA	Calibration Mode	Logic Elements (Logic, Adders, Flipflops, etc.)	Memory Blocks	DSP Blocks	Efinity® Version ⁽¹⁾
Ti60 F225 C4	AXI PLL Auto	2,033/60,800 (3.3%)	25/256 (9.7%)	0/160 (0%)	2023.2
	Native PLL Auto	1,730/60,800 (2.8%)	22/256 (8.5%)	0/160 (0%)	

Release Notes

You can refer to the IP Core Release Notes for more information about the IP core changes. The IP Core Release Notes are available in the [Efinity page of the Support Center](#) under each Efinity software release version.



Note: You must be logged in to the Support Center to view the IP Core Release Notes.

⁽¹⁾ Using Verilog HDL.

Functional Description

The HyperRAM Controller core supports three calibration modes:

- *PLL auto calibration*—Utilizes the Titanium and Topaz PLL's dynamic shifting to calibrate incoming data from the HyperRAM.
- *PLL manual calibration*—Allows you to manually calibrate the PLL phase settings used to capture the incoming data without using the auto calibration logic.
- *Soft logic calibration*—Uses extra logic to calibrate incoming data from the HyperRAM.



Note: The PLL calibration mode HyperRAM Controller I/O's path is not impacted by design recompilations, therefore Efinix recommends that you use PLL auto calibration for Titanium and Topaz FPGAs.

Figure 1: HyperRAM Controller PLL Calibration Block Diagram

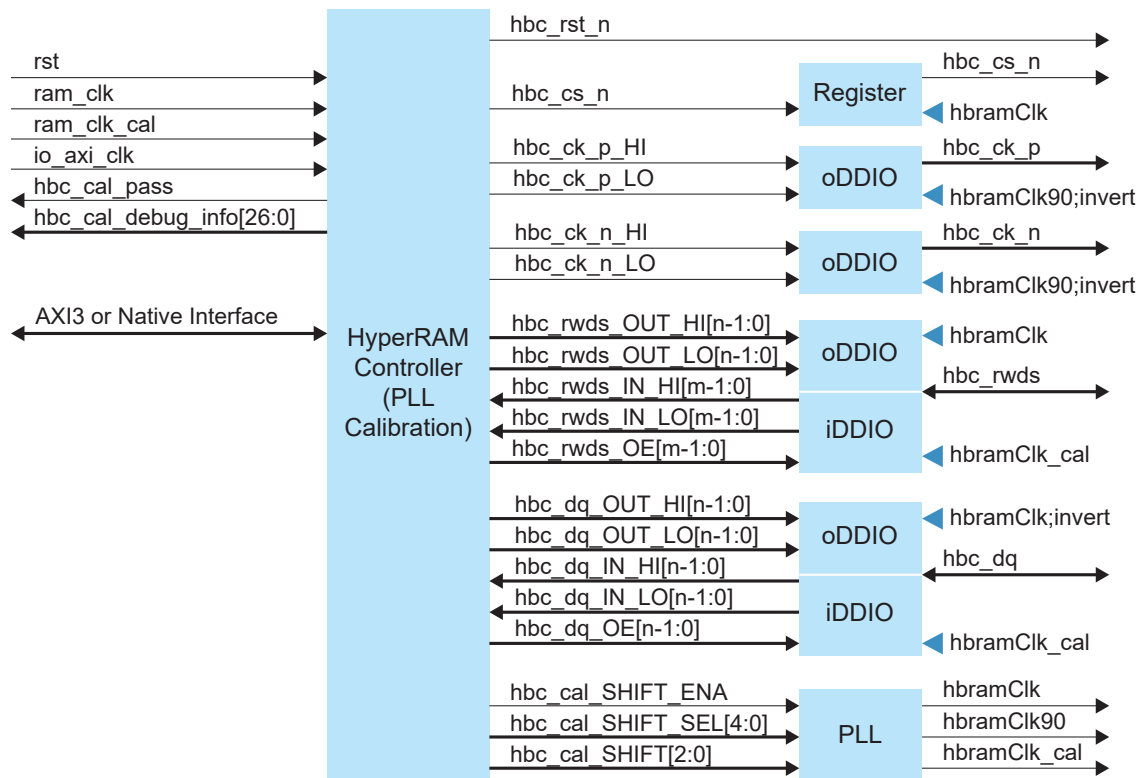


Figure 2: HyperRAM Controller PLL Manual Calibration Block Diagram

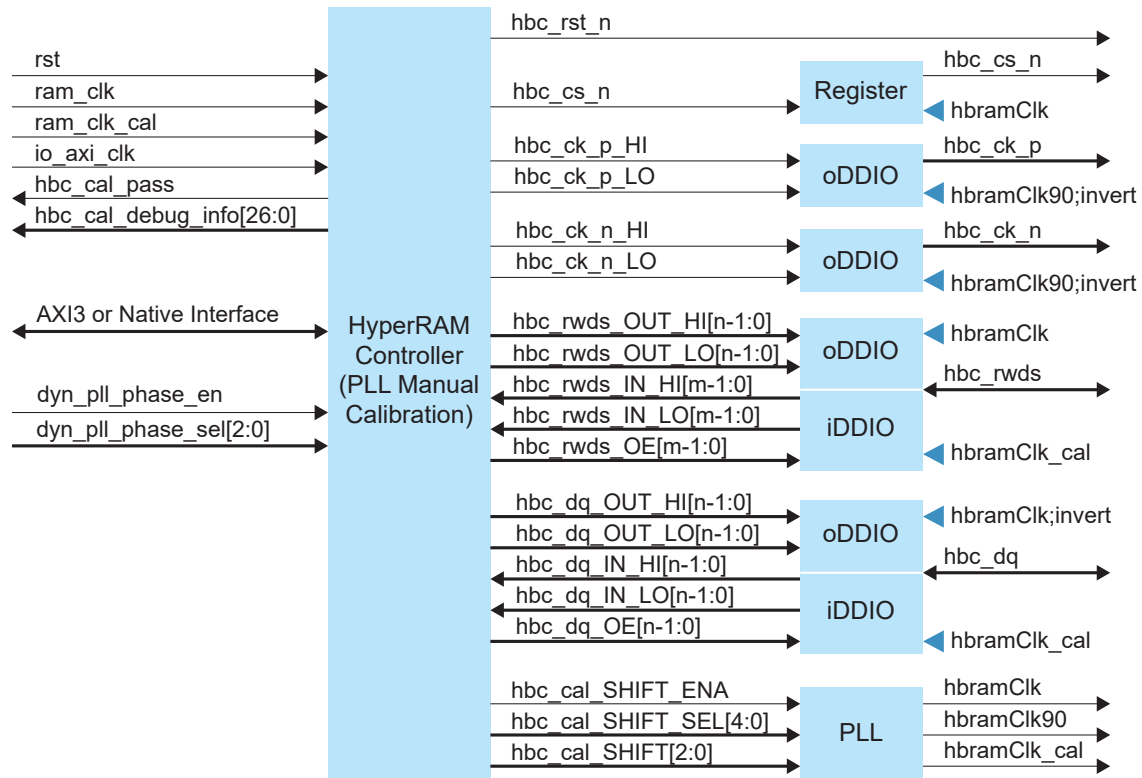
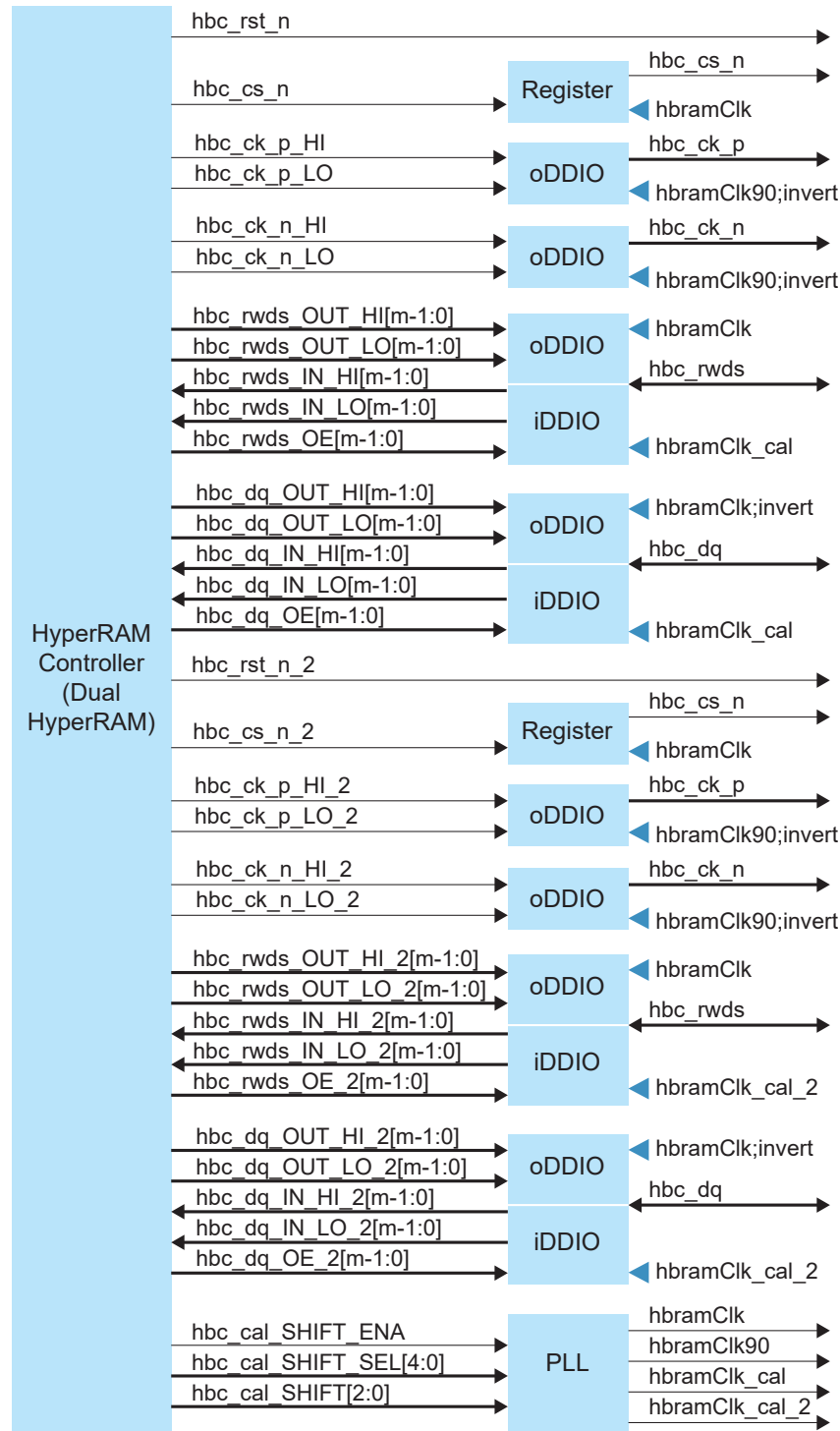


Figure 3: HyperRAM Controller Dual HyperRAM Block Diagram

When the Dual HyperRAM is enabled, you can connect two HyperRAM to the HyperRAM Controller Core.



Ports

Table 3: HyperRAM Controller Ports

Port	Direction	Description
hbc_cs_n	Output	Bus transactions are initiated with a logic high-to-low transition. Bus transactions are terminated with a logic low-to-high transition. The master device has a separate CS# signal for each slave.
rst	Input	Core asynchronous reset, active high.
ram_clk	Input	RAM operating clock.
ram_clk_cal	Input	RAM calibration clock. Used for PLL auto and manual calibration only.
ram_clk_cal_2	Input	RAM 2 calibration clock. Used for Dual HyperRAM with PLL auto calibration only.
hbc_rst_n	Output	When logic low, the slave device self-initializes and returns to the standby state. RWDS and DQ signals are placed into the high-Z state when the RESET# signal is low.
hbc_cal_pass	Output	Indicates calibration is passing.
hbc_cs_n_2	Output	Chip select for second HyperRAM.
hbc_rst_n_2	Output	When logic low, the slave device self-initializes and returns to the standby state. RWDS and DQ signals are placed into the high-Z state when the RESET# signal is low.
hbc_ck_p_HI_2	Output	Differential Clock: Command, address, and data information is output with respect to the crossing of the CK and CK# signals for second HyperRAM. Single Ended Clock: CK# is not used, only a single ended CK is used. The clock is not required to be free-running.
hbc_ck_p_LO_2	Output	
hbc_ck_n_HI_2	Output	
hbc_ck_n_LO_2	Output	
hbc_dq_OUT_HI_2 [n-1:0]	Output	DQ output ports for command, address, and data for second HyperRAM. $n = 8$ (x8 mode), 16 (x16 mode)
hbc_dq_OUT_LO_2 [n-1:0]	Output	
hbc_dq_OE_2 [n-1:0]	Output	DQ output enable port for second HyperRAM. $n = 8$ (x8 mode), 16 (x16 mode)
hbc_dq_IN_HI_2 [n-1:0]	Input	DQ input ports for data for second HyperRAM. $n = 8$ (x8 mode), 16 (x16 mode)
hbc_dq_IN_LO_2 [n-1:0]	Input	
hbc_rwds_OUT_HI_2 [m-1:0]	Output	RWDS output ports for data mask during write operation for second HyperRAM. $m = 1$ (x8 mode), 2 (x16 mode)
hbc_rwds_OUT_LO_2 [m-1:0]	Output	
hbc_rwds_OE_2 [m-1:0]	Output	RWDS output enable port for second HyperRAM. $m = 1$ (x8 mode), 2 (x16 mode)

Port	Direction	Description														
hbc_rwds_IN_HI_2 [<i>m</i> -1:0]	Input	RWDS input enable port for second HyperRAM. <i>m</i> = 1 (x8 mode), 2 (x16 mode)														
hbc_rwds_IN_LO_2 [<i>m</i> -1:0]	Input															
hbc_cal_debug_info [26:0]	Output	<table><tr><th>Bit</th><th>Description</th></tr><tr><td>26:19</td><td>Delay steps that received matched calibration data by second HyperRAM. One-hot encoded Bit 0: Delay step 0 received matched data Bit 1: Delay step 1 received matched data Bit 2: Delay step 2 received matched data Bit 3: Delay step 3 received matched data Bit 4: Delay step 4 received matched data Bit 5: Delay step 5 received matched data Bit 6: Delay step 6 received matched data Bit 7: Delay step 7 received matched data</td></tr><tr><td>18:16</td><td>Locked down PLL phase shifted step for second HyperRAM.</td></tr><tr><td>15:8</td><td>Delay steps that received matched calibration data. One-hot encoded. Bit 0: Delay step 0 received matched data Bit 1: Delay step 1 received matched data Bit 2: Delay step 2 received matched data Bit 3: Delay step 3 received matched data Bit 4: Delay step 4 received matched data Bit 5: Delay step 5 received matched data Bit 6: Delay step 6 received matched data Bit 7: Delay step 7 received matched data</td></tr><tr><td>7:5</td><td>Locked down PLL phase shifted step</td></tr><tr><td>1</td><td>Calibration pass</td></tr><tr><td>0</td><td>Calibration done</td></tr></table>	Bit	Description	26:19	Delay steps that received matched calibration data by second HyperRAM. One-hot encoded Bit 0: Delay step 0 received matched data Bit 1: Delay step 1 received matched data Bit 2: Delay step 2 received matched data Bit 3: Delay step 3 received matched data Bit 4: Delay step 4 received matched data Bit 5: Delay step 5 received matched data Bit 6: Delay step 6 received matched data Bit 7: Delay step 7 received matched data	18:16	Locked down PLL phase shifted step for second HyperRAM.	15:8	Delay steps that received matched calibration data. One-hot encoded. Bit 0: Delay step 0 received matched data Bit 1: Delay step 1 received matched data Bit 2: Delay step 2 received matched data Bit 3: Delay step 3 received matched data Bit 4: Delay step 4 received matched data Bit 5: Delay step 5 received matched data Bit 6: Delay step 6 received matched data Bit 7: Delay step 7 received matched data	7:5	Locked down PLL phase shifted step	1	Calibration pass	0	Calibration done
		Bit	Description													
		26:19	Delay steps that received matched calibration data by second HyperRAM. One-hot encoded Bit 0: Delay step 0 received matched data Bit 1: Delay step 1 received matched data Bit 2: Delay step 2 received matched data Bit 3: Delay step 3 received matched data Bit 4: Delay step 4 received matched data Bit 5: Delay step 5 received matched data Bit 6: Delay step 6 received matched data Bit 7: Delay step 7 received matched data													
		18:16	Locked down PLL phase shifted step for second HyperRAM.													
		15:8	Delay steps that received matched calibration data. One-hot encoded. Bit 0: Delay step 0 received matched data Bit 1: Delay step 1 received matched data Bit 2: Delay step 2 received matched data Bit 3: Delay step 3 received matched data Bit 4: Delay step 4 received matched data Bit 5: Delay step 5 received matched data Bit 6: Delay step 6 received matched data Bit 7: Delay step 7 received matched data													
		7:5	Locked down PLL phase shifted step													
		1	Calibration pass													
		0	Calibration done													
hbc_ck_p_HI	Output	Differential Clock: Command, address, and data information is output with respect to the crossing of the CK and CK# signals. Single Ended Clock: CK# is not used, only a single ended CK is used. The clock is not required to be free-running.														
hbc_ck_p_LO	Output															
hbc_ck_n_HI	Output															
hbc_ck_n_LO	Output															
hbc_dq_OUT_HI [n-1:0]	Output	DQ output ports for command, address, and data. <i>n</i> = 8 (x8 mode), 16 (x16 mode)														
hbc_dq_OUT_LO [n-1:0]	Output															
hbc_dq_OE [n-1:0]	Output	DQ output enable port. <i>n</i> = 8 (x8 mode), 16 (x16 mode)														
hbc_dq_IN_HI [n-1:0]	Input	DQ input ports for data. <i>n</i> = 8 (x8 mode), 16 (x16 mode)														
hbc_dq_IN_LO[n-1:0]	Input															

Port	Direction	Description
hbc_rwds_OUT_HI [m-1:0]	Output	RWDS output ports for data mask during write operation. $m = 1$ (x8 mode), 2 (x16 mode)
hbc_rwds_OUT_LO [m-1:0]	Output	
hbc_rwds_IN_HI [m-1:0]	Input	RWDS input ports for latency indication, also center-aligned reference strobe for read data. $m = 1$ (x8 mode), 2 (x16 mode)
hbc_rwds_IN_LO [m-1:0]	Input	RWDS input ports for latency indication, also center-aligned reference strobe for read data. $m = 1$ (x8 mode), 2 (x16 mode) Applicable in PLL auto and manual calibration only.
hbc_rwds_OE [m-1:0]	Output	RWDS output enable port. $m = 1$ (x8 mode), 2 (x16 mode)
hbc_cal_SHIFT_ENA	Output	Enables PLL dynamic shifting. Applicable in PLL auto and manual calibration only.
hbc_cal_SHIFT_SEL[4:0]	Output	Selects PLL output. Applicable in PLL auto and manual calibration only.
hbc_cal_SHIFT[2:0]	Output	Delay steps value. Applicable in PLL auto and manual calibration only.
dyn_pll_phase_en	Input	Enable PLL phase adjustment. Used for PLL manual calibration only. 1'b0: Disable 1'b1: Enable
dyn_pll_phase_sel [2:0]	Input	PLL phase adjustment. 8 steps of the PLL dynamic phase shift settings. Used for PLL manual calibration only.

Table 4: HyperRAM Controller AXI Ports

Port	Direction	Description
io_axi_clk	Input	AXI interface operating frequency.
io_arw_valid	Input	Indicates that the channel is signaling a valid write/read address and control information.
io_arw_ready	Output	Indicates that the controller is ready to accept an address and associated control signals.
io_arw_payload_addr [31:0]	Input	The write address gives the address of the first transfer in a write/read burst transaction.  Note: Bit io_arw_payload_addr [31] is reserved. Set this bit to zero.
io_arw_payload_id [7:0]	Input	Identification tag for the write/read address group of signals.  Note: HyperRAM AXI does not support address reordering or data interleaving.
io_arw_payload_len [7:0]	Input	Burst length. Indicates the exact number of transfers in a burst. Determines the number of data transfers associated with the address. Effective burst length = io_arw_payload_len + 1

Port	Direction	Description
io_arw_payload_size [2:0]	Input	Burst size. Indicates the size of each transfer in the burst. 3'b000: 1 3'b001: 2 3'b010: 4 3'b011: 8 3'b100: 16 3'b101: 32 3'b110: 64 3'b111: 128
io_arw_payload_burst [1:0]	Input	Burst type. The burst type and the size information, determines how the address for each transfer within the burst is calculated. The controller does not support fixed burst. 2'b01: Linear burst 2'b00, 2'b10: Wrap burst
io_arw_payload_lock	Input	Reserved.
io_arw_payload_write	Input	Indicates the channel is accepting a write or read transfer. 1'b0: Read 1'b1: Write
io_w_payload_id [7:0]	Input	ID tag of the write data transfer. Information is not used by the controller.  Note: HyperRAM Controller core's AXI channel does not support address reordering or data interleaving.
io_w_valid	Input	Write valid. Indicates that valid write data and strobes are available.
io_w_ready	Output	Write ready. Indicates that the slave can accept the write data.
io_w_payload_data [n-1:0]	Input	Write data. $n = \text{AXI_DBW}$
io_w_payload_strb [n-1:0]	Input	Write strobes. Indicates which byte lanes hold valid data. There is one write strobe bit for each eight bits of the write data bus. $n = \text{AXI_DBW}/8$
io_w_payload_last	Input	Write last. Indicates the last transfer in a write burst.
io_b_valid	Input	Write response valid. Indicates that the channel is signaling a valid write response.
io_b_ready	Output	Response ready. Indicates that the master can accept a write response.
io_b_payload_id [7:0]	Output	Response ID tag. ID tag of the write response.  Note: HyperRAM Controller core's AXI channel does not support address reordering or data interleaving.
io_r_valid	Output	Read address valid. Indicates that the channel is signaling valid read address and control information.
io_r_ready	Input	Read ready. Indicates that the master can accept the read data and response information.
io_r_payload_data	Output	Read data.
io_r_payload_id [7:0]	Output	Read ID tag. Identification tag for the read data group of signals generated by the controller.

Port	Direction	Description
io_r_payload_resp [1:0]	Output	Read response. Indicates the status of the read transfer. This controller only responds 'b00 or OKAY.
io_r_payload_last	Output	Read last. Indicates the last transfer in a read burst.

Table 5: HyperRAM Controller Native Ports

Port	Direction	Description
native_ram_rdwr	Input	HyperRAM write/read control. 1'b0 : Target for write 1'b1 : Target for read
native_ram_en	Input	Initiate a single pulse to trigger controller write/read to HyperRAM. For write operation, you must ensure that the data stored in the write buffer fulfills the configured burst length requirement before pulsing the signal. After pulsing the signal, monitor the native_ctrl_idle signal transit into low state to indicate the controller has started to process the request.
native_ram_burst_len [11:0]	Input	HyperRAM transaction burst length. You can dynamically change the burst length to maximize the transfer efficiency while still fulfilling the memory CS# maximum low time, 4 μ s (typical). The burst length number must not exceed the CS# maximum low time which is governed by the HyperRAM specification. FIFO Depth Size = ((DQ Width * 2) / Data width) * native_ram_burst_len For example: 512 burst length with x16 RAM at 200 MHz will have approximately 2.6 μ s CS# low time.  Note: Refer to Ti60/Ti35 data sheet for the t_{CSM} specification (CS# low) if you are working with the HyperRAM in F100S3F2.
native_ram_address [31:0]	Input	HyperRAM write/read address. The address width depends on the memory density. native_ram_address [31] = 1: Linear burst (default) 0: Reserved native_ram_address [30] = 1: Reserved 0: Memory space (default)
native_wr_en	Input	Buffer write enable. Assert this signal when you want to place the write data into the write buffer. You can fill-in the write buffer at any time if the write buffer is not full indicated by the native_wr_buf_ready signal.
native_wr_data [n-1:0]	Input	Write data is placed in the write buffer before the native_ram_en signal is pulsed. $n = AXI_DBW$
native_wr_datamask [n/8-1:0]	Input	Write data mask. Set all to logic low if you do not use the write data mask. Otherwise, drive the signal per data byte (8 bits) granularity. $n = AXI_DBW$
native_rd_data[n-1:0]	Output	Read data width. $n = AXI_DBW$

Port	Direction	Description
native_rd_valid	Output	Read valid indicator. Logic high indicates that the returning data is valid.
native_ctrl_idle	Output	Logic high indicates that the controller is in idle mode. After you issue the native_ram_en signal, this signal will take multiple cycles to respond due to the clock domain crossing operation. Ensure that the signal is deasserted after issuing a write or read operation.
native_wr_buf_ready	Output	Write buffer availability. Logic low indicates that the write buffer is full and the write buffer ignores any incoming write data.

Table 6: Periodic Calibration User Ports

Port	Direction	Description
prd_cal_req	Output	Periodic calibration request signal. Serves as status pin when On-Demand Calibration is selected. Domain: io_axi_clk / native_clk
prd_cal_ack	Input	Periodic calibration acknowledge signal. Used to trigger calibration when On-Demand Calibration is selected. Domain: io_axi_clk / native_clk

AXI User Interface

AXI Addressing

The AXI interface uses byte addressing, where each address holds 8-bit data. However, for HyperRAM, the addressing depends on the RAM bit width. For x8 mode, each address holds 16-bit data and for x16 mode, each address holds 32-bit data. The HyperRAM Controller core converts the AXI address to match the HyperRAM address.

The HyperRAM Controller core's AXI Interface does not support address reordering and data interleaving. The write ID must follow the order sent through the AXI's address channel. The write respond channel responds to the IDs with the sequence input from the user through the AXI's address channel.

Table 7: AXI Address Conversion Ratio

Mode	Divider Ratio
Two x16 (x32)	8
Two x8 (x16)	4
X16	4
X8	2

When using a 512 Mb HyperRAM, user must not cross the die boundaries when performing burst operations. Due to the HyperRAM's Dual-Die-Package, burst operation can only be performed with one die. Refer to **Table 8: 512 Mb HyperRAM AXI Address Boundary** on page 14 for address boundaries details of the AXI Interface.

Table 8: 512 Mb HyperRAM AXI Address Boundary

Mode	AXI Address Boundary (Die 0)	AXI Address Boundary (Die 1)
Two x16 (x32)	h0000 0000 ~ h03FF FFF8	h0400 0000 ~ h07FF FFF8
Two x8 (x16)	h0000 0000 ~ h03FF FFFC	h0400 0000 ~ h07FF FFFC
x16	h0000 0000 ~ h01FF FFFC	h0200 0000 ~ h03FF FFFC
x8	h0000 0000 ~ h01FF FFFE	h0200 0000 ~ h03FF FFFE

Unaligned Address Access

The AXI interface does not support unaligned address transfers. Instead, the AXI data width determines the address access.

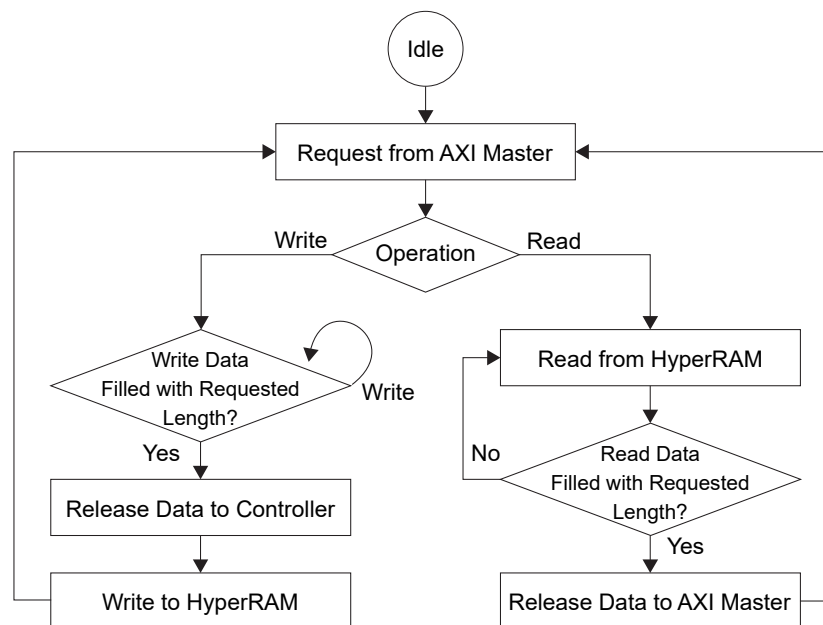
AXI Bit Width	Base Address Multiplier
256	32
128	16
64	8
32	4

For example, if you request h000012F8 on a 128-bits data width, the HyperRAM Controller core aligns the address to the nearest base address, which is h000012F0.

AXI Read and Write Operation

The following flow diagram explains the AXI read and write operation in the HyperRAM Controller core.

Figure 4: AXI3 Operation Flow Diagram



Native User Interface

The native user interface allows you to access memory in a more direct way, for example, you can issue an actual RAM address, controlling write or read operations, set RAM burst transaction length, or perform write mask as per the HyperRAM supported features. Asynchronous data FIFO is implemented between the user logic and the controller to facilitate the clock crossing between two different domains.

The native user interface uses word addressing which is according to the width of the HyperRAM, and not the native data width. For example, on an x16DQ HyperRAM, each address holds 32-bit data. Subsequently, on a x8DQ HyperRAM, each address holds 16-bit data.

This also applies to burst length as well. For example, on a x16DQ HyperRAM, one burst length is fixed to 32-bit data, a burst length of 10 will transfer a total of $10 \times 32\text{-bit} = 320$ bits of data to the HyperRAM. On a x8DQ HyperRAM, one burst length is fixed to 16-bit data.

For write operations, you need to set the write buffer depth as per the targeted RAM write length. Since the write operation is continuous, you need to fill-up the data buffer before issue a write trigger. You can store the data in the buffer at any time if the buffer is not full. For example, if the targeted write length (`native_ram_burst_len`) is 256-bit:

- If the native data width is 32-bit, the write buffer depth must be at least 256
- If the native data width is 64-bit, the write buffer depth must be at least 128

You calculate the write FIFO depth size with the following formula:

Write FIFO Depth Size = $((\text{DQ Width} \times 2) / \text{Data width}) \times \text{native_ram_burst_len}$



Note: A single write or read operation should not exceed the 4 us time memory CS# maximum low time. See [Table 5: HyperRAM Controller Native Ports](#) on page 12 for `native_ram_burst_len` detailed description.

For read operations, the data returns to the user logic side with `native_rd_valid` set high whenever the data is available.

In both write and read operations, you should always monitor the controller status signal, `native_ctrl_idle`. This signal should go high after you issue a transaction to ensure the controller has carry out the request. You must not perform any read or write burst crossing the die boundary. Refer to [Table 9: 512 Mb HyperRAM Native Address Boundary](#) on page 16 for value on the die boundaries when using the 512 Mb RAM.

Table 9: 512 Mb HyperRAM Native Address Boundary

Mode	Native Address Boundary (Die 0)	Native Address Boundary (Die 1)
Two x16 (x32)	h8000 0000 ~ h807F FFF8	h8080 0000 ~ h80FF FFFF
Two x8 (x16)	h8000 0000 ~ h80FF FFFF	h8100 0000 ~ h81FF FFFF
x16	h8000 0000 ~ h807F FFFF	h8080 0000 ~ h80FF FFFF
x8	h8000 0000 ~ h80FF FFFF	h8100 0000 ~ h81FF FFFF

PLL Auto Calibration Flow

Upon power-up, the AXI wrapper block `io_arw_ready` is deasserted to avoid taking any requests from the AXI master before calibration completes. The calibration master module sends the default setting of configuration register 0 and configuration register 1 to the HyperRAM and follows by writing 512 bytes of data on address `'h00000000`. The calibration slave module starts reading data from the HyperRAM continuously and adjusts the delay by providing different dynamically shifted clock (incrementing from 0 to 7, on recommended PLL settings each step is 45° (degree) delayed). This value changes the:

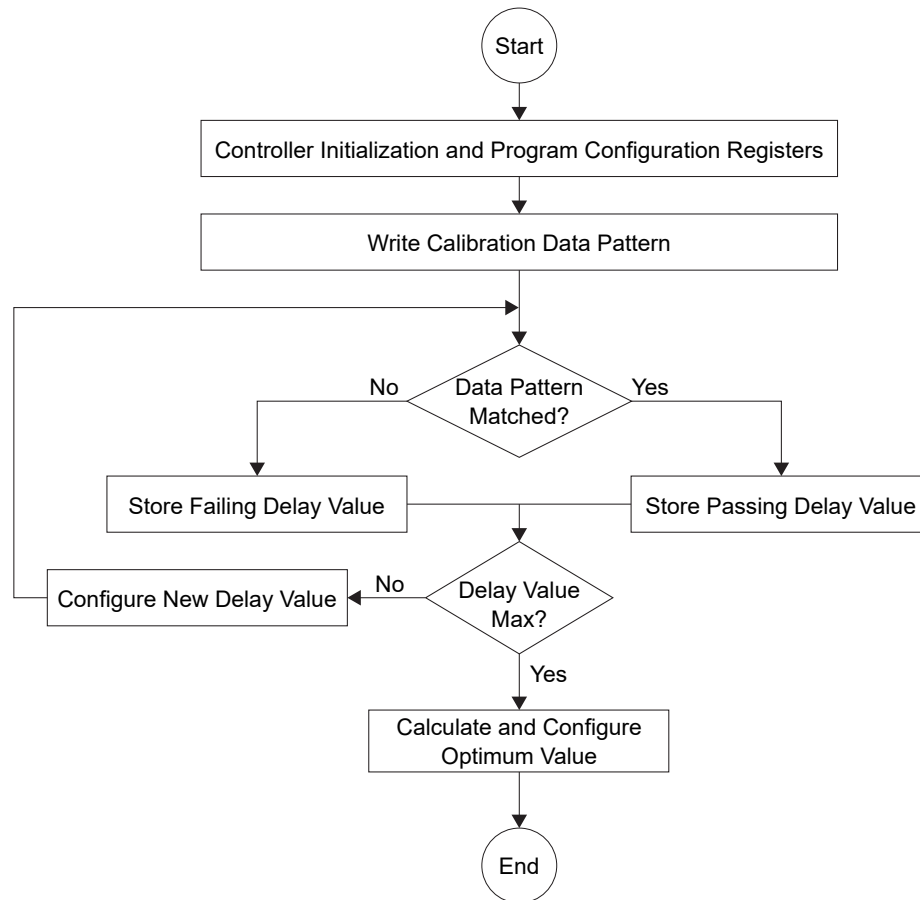
- *hbramClk_cal phase (PLL auto calibration)*—The `hbramClk_cal` is used to register the input of data and read/write data strobe (RWDS) in double-data rate I/O (DDIO) mode.
- *Mux delay module (soft logic calibration)*—The mux delay module is a chain of mux to emulate the delay path to DQ and RWDS.

Once the calibration slave module receives matched data from the HyperRAM, it stores the delay value, you can refer the delay values from `hbc_cal_debug_info[15:8]`. It keeps testing by incrementing the delay value and eventually chooses the optimum value to lock on. The HyperRAM Controller core asserts `hbc_cal_pass` high once to exit calibration and assert `io_arw_ready` signal.

If there is no matching data from the HyperRAM after 8 (eight) delays, calibration is failed and `hbc_cal_pass` remains 0 with `hbc_cal_done` becomes 1.

The HyperRAM Controller core performs a one-off initial calibration. It is recommended to perform periodic recalibration by resetting the HyperRAM Controller core to maintain the optimal settings on the interface between the controller and the memory, or you may experience errors or unexpected behavior.

Figure 5: Calibration Flow Diagram

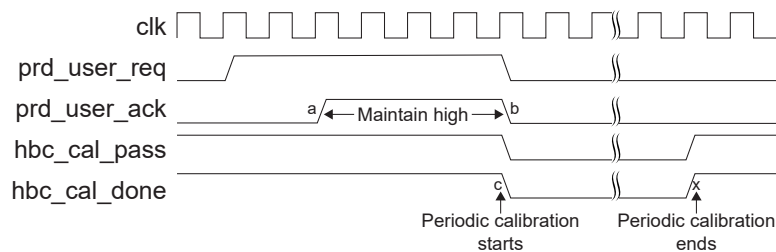


Periodic Recalibration Flow

Besides the one-off auto calibration flow, you can use the periodic calibration feature provided by the HyperRAM Controller core to recalibrate the HyperRAM at a set time interval. The time interval can be configured through the IP Configuration GUI.

When it is time, you can choose to start calibration through the user signal provided. Your existing data that was written into the HyperRAM is not affected by periodic calibration. When the periodic calibration starts, any transaction at the user interface is paused and resumes after the calibration is done. The following figure illustrates the periodic recalibration flow waveform.

Figure 6: Periodic Recalibration Flow Waveform



On-Demand Calibration

On-demand calibration is also a sub-feature that can be enabled together with PLL auto calibration and periodic recalibration. This feature allows you to perform calibration whenever you need it, `prd_cal_req` changes to become a status pin. When `prd_cal_req==1`, it indicates the timer has expired and goes into calibration soon as shown in **Figure 8: Timer Trigger Recalibration** on page 20. To initiate recalibration manually at any time, you need to assert `prd_cal_ack` and hold `prd_cal_ack` high until `hbc_cal_pass` becomes low as shown in **Figure 7: User Trigger Recalibration on Demand** on page 20. Do not deassert `prd_cal_ack` once you have asserted it.

Figure 7: User Trigger Recalibration on Demand

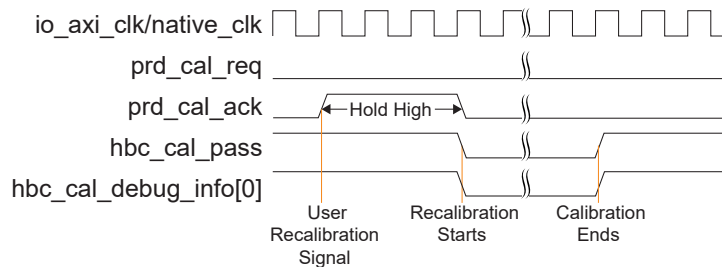
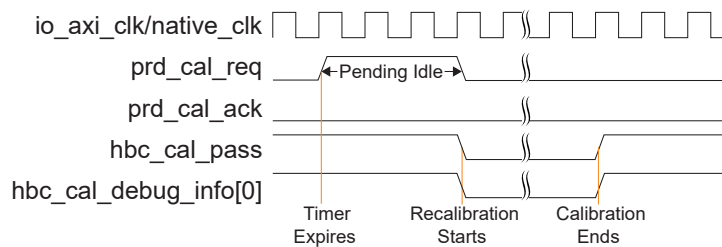


Figure 8: Timer Trigger Recalibration



The timer reset selection is provided when the on-demand calibration is enabled. When the timer reset is enabled, the timer is reset when you trigger the calibration through `prd_cal_ack`. Disabling the timer reset prevents the timer from resetting, causing it to continue counting even after you have manually trigger the calibration.

PLL Manual Calibration Flow

In PLL Manual calibration mode, you need to manually set the PLL phase shift by changing the `dyn_pll_phase_sel [2:0]`. The value of `dyn_pll_phase_sel` and `dyn_pll_phase_en` must be sustained for 4 or more clock cycles to change the phase shift on the PLL clock. You can leverage on the PLL Auto calibration mode to search for the optimum settings before applying the settings in your design. Refer to **HyperRAM Controller Example Design** on page 37. However, the fixed settings you use in the PLL Manual calibration mode may not be the optimum setting when there are variations in voltage and temperature.

The PLL Manual Filter parameter filters out incorrectly sampled data. When enabled, if the `dyn_pll_phase_sel [2:0]` is outside the valid window, the user will not observe any data output from the HyperRAM Controller core's user channel. This signifies that the selected `dyn_pll_phase_sel [2:0]` is not viable. Users can switch off this feature to allow the data sampled outside the ideal window to be output by the HyperRAM Controller core.

HyperRAM Controller Operation

The following waveforms describes timing sequence of signals between the HyperRAM Controller and interface connections during write and read operations.

Hyperbus Interface

Figure 9: Hyperbus Interface Write Operation Waveform

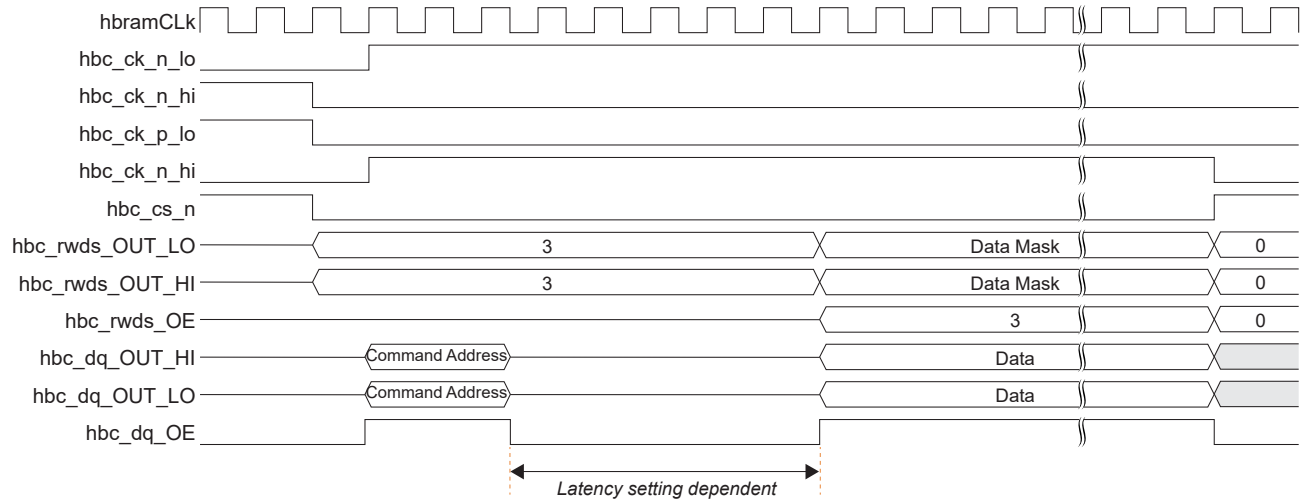
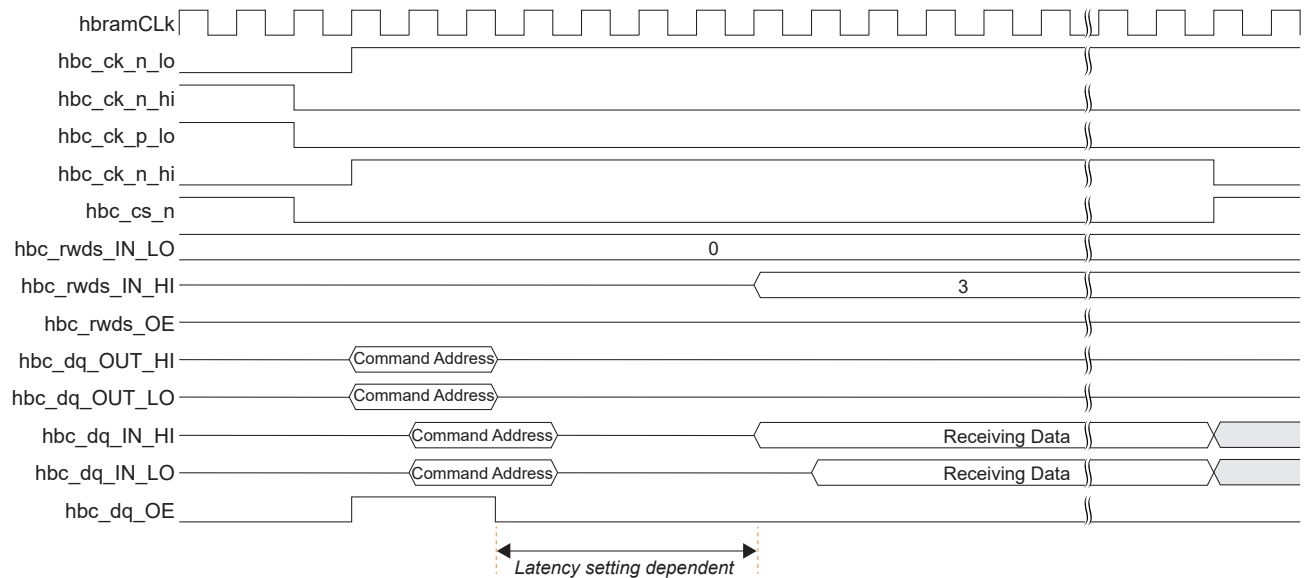


Figure 10: Hyperbus Interface Read Operation Waveform



AXI Interface



Note: The HyperRAM controller uses a half-duplex AXI Protocol. You need to send a read or write request through the `io_arw_payload_write` together with the address to initiate a read or write operation.

Figure 11: AXI Interface Write Operation

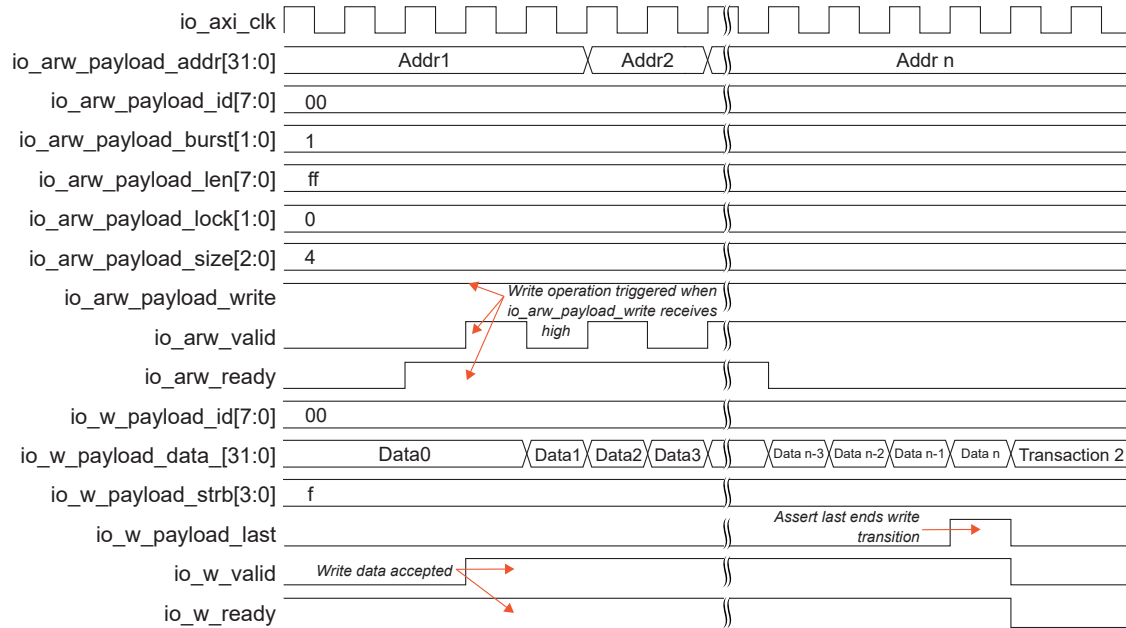
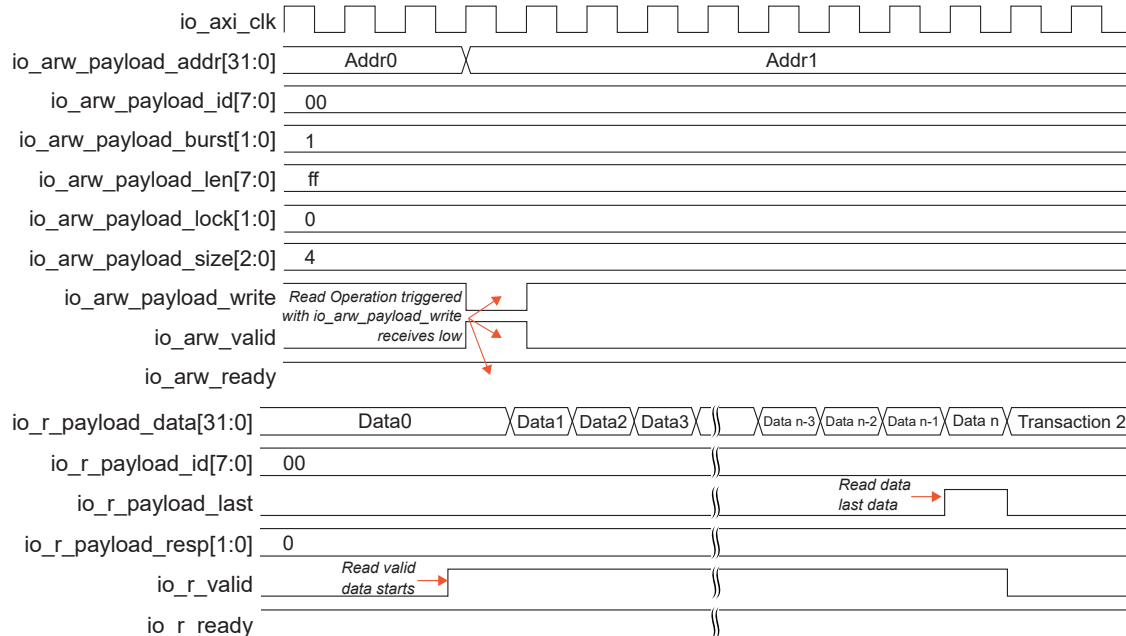


Figure 12: AXI Interface Read Operation



Native Interface



Note: Native interface mode only supports linear mode operation and requires the `native_ram_address[31]` to always be set to high.

Figure 13: Native Interface Write Operation Waveform

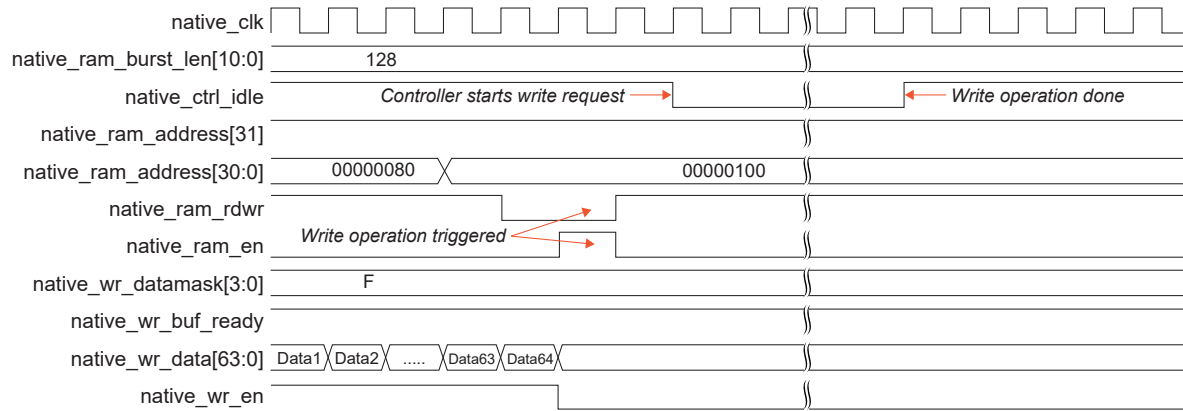
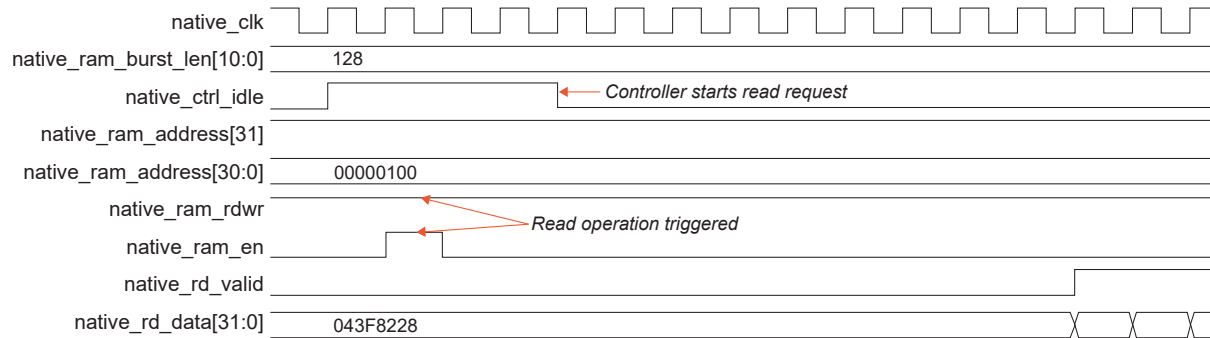


Figure 14: Native Interface Read Operation Waveform



Dynamic Reconfiguration for Configuration Register

The HyperRAM Controller core allows you to perform configuration register read or write to the HyperRAM. To use the dynamic reconfiguration feature, select **Enable** on **Dynamic Reconfiguration** in Memory Register tab. This is only available with IP version v6.12 or above.

To perform dynamic reconfiguration, you need to write the specific address into the user interface, this address is different depending on the type of HyperRAM and the type of user interface (AXI or Native). You can refer to [Table 10: AXI Interface Configuration Register Read/Write Address](#) on page 25 and [Table 11: Native Interface Configuration Register Read/Write Address](#) on page 25.

Table 10: AXI Interface Configuration Register Read/Write Address

Type of HyperRAM Configuration	Register Write Address (io_arw_payload_write == 1)		Register Read Address (io_arw_payload_write == 0)	
	CR0	CR1	CR0	CR1
x8	32'h80001000	32'h80001002	32'h80001000	32'h80001002
x16	32'h80002000	32'h80002004	32'h80002000	32'h80002004
2x8 (Dual HyperRAM)	32'h80002000	32'h80002004	32'h80002000	32'h80002004
2x16 (Dual HyperRAM)	32'h80004000	32'h80004008	32'h80004000	32'h80004008

Table 11: Native Interface Configuration Register Read/Write Address

Type of HyperRAM Configuration	Register Write Address (native_ram_rdwr == 0)		Register Read Address (native_ram_rdwr == 1)	
	CR0	CR1	CR0	CR1
x8	32'hC0000800	32'hC0000801	32'hC0000800	32'hC0000801
x16	32'hC0000800	32'hC0000801	32'hC0000800	32'hC0000801
2x8	32'hC0000800	32'hC0000801	32'hC0000800	32'hC0000801
2x16	32'hC0000800	32'hC0000801	32'hC0000800	32'hC0000801

The length of the data sent must be the smallest length for the data width. Refer to **Table 12: Value of length for Configuration Register Read/Write** on page 26 for the length value needed for each configuration.

Table 12: Value of length for Configuration Register Read/Write

User Interface	Dual HyperRAM	Data Width	Length	Burst Type
AXI	No	256	0	01 (Linear burst)
		128	0	01 (Linear burst)
		64	0	01 (Linear burst)
		32	0	01 (Linear burst)
Native	No	128	4	Linear burst
		64	2	Linear burst
		32	1	Linear burst
	Yes	128	2	Linear burst
		64	1	Linear burst

When performing register write, the content that is written to `io_w_payload_data` or `native_wr_data` must follow the pattern in **Table 13: Configuration Register Read Data Example** on page 26.



Note: The input bit pattern is different for single RAM and dual RAM.

Table 13: Configuration Register Read Data Example

HyperRAM Width	Expected Read Data (User Interface)	Example Read Data User Width = 64 with default CR0 content
x8	{User Width/16 {CR[15:0]}}	64'h 8f2f 8f2f 8f2f 8f2f
x16	{User Width/32 {'h00, CR[15:8], 'h00, CR[7:0]}}	64'h 008f 002f 008f 002f

Table 14: Data Pattern mapped to Configuration Register Bits

Type of HyperRAM Configuration	User Input on User Channel (CR = Configuration Register)	Example Write Data User Width = 64 with deep power down
x8	{User_Width/16 { CR[15:8], CR[7:0], CR[15:8], CR[7:0] }}	64'h 0f2f 0f2f 0f2f 0f2f
x16	{User_Width/32 {CR[15:8], CR[15:8], CR[7:0], CR[7:0] }}	64'h 000f 002f 000f 002f

Figure 15: Example of AXI Interface's Configuration Register Write on page 27 and **Figure 16: Example of Native Interface's Configuration Register Write** on page 27 show an example of register write to trigger hybrid sleep. To exit hybrid sleep or deep power down, request a new transaction, and the HyperRAM Controller core IP handles the exit sequence.

Figure 15: Example of AXI Interface's Configuration Register Write

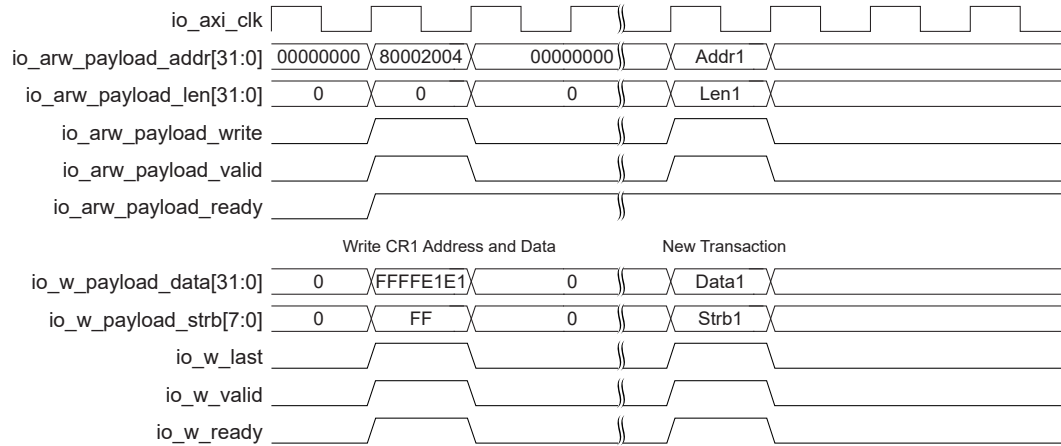
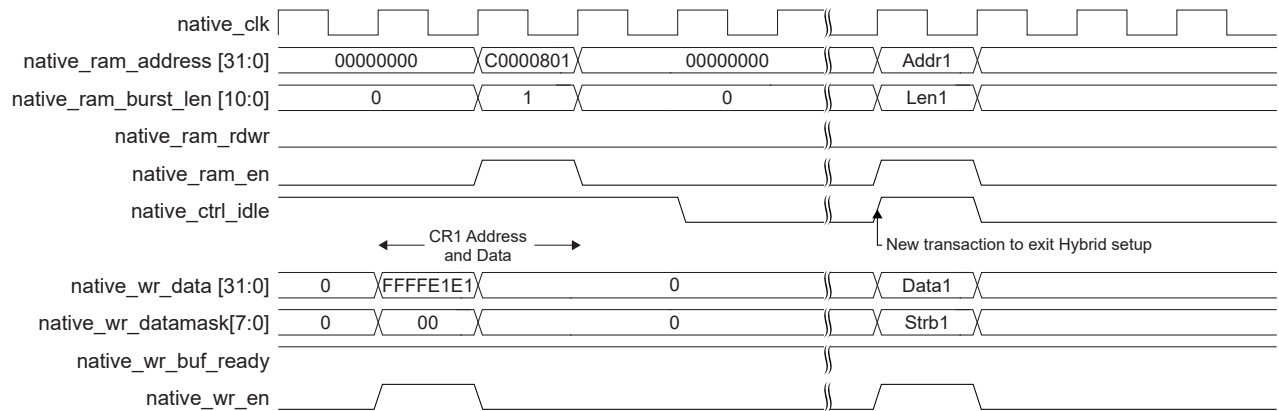


Figure 16: Example of Native Interface's Configuration Register Write



Configuration Register

The HyperRAM Controller core supports many features. However, different HyperRAM models come with different sets of features. Efinix's HyperRAM Controller core only supports the majority of features from Winbond's W958D6NKY HyperRAM, which is available in SIP package on the Ti60F100S3F2 FPGA chip. Refer to **Table 15: Configuration Register 0 Bit Assignments** on page 28 and **Table 16: Configuration Register 1 Bit Assignments** on page 29 for supported and unsupported features.

Configuration Register 0

Table 15: Configuration Register 0 Bit Assignments

CR0 Bit	Function	Settings (Binary)
[15]	Deep Power Down Enable	1b: Normal operation (default) 0b: Writing 0 to CR0[15] causes the device to enter deep power down (DPD) HyperRAM automatically sets the value of CR0[15] to 1 after exiting the DPD.  Warning: Contents of configuration register are lost when using deep power down, rewriting the registers with the proper setting is needed upon exiting deep power down.
[14:12]	Drive Strength	000b: 34 ohms (default) 001b: 115 ohms 010b: 67 ohms 011b: 46 ohms 100b: 34 ohms 101b: 27 ohms 110b: 22 ohms 111b: 19 ohms
[11:8]	Reserved	1b: Reserved (default) When writing this register, this bit should be kept 1b.
[7:4]	Initial Latency	0000b: 5 Clock Latency at 133MHz maximum frequency 0001b: 6 Clock Latency at 166MHz maximum frequency 0010b: 7 Clock Latency at 200MHz maximum frequency (default) 0010b: 7 Clock Latency at 250MHz maximum frequency 0011b: Reserved 0100b: Reserved ... 1101b: Reserved 1110b: 3 Clock Latency at 85MHz maximum frequency 1111b: 4 Clock Latency at 104MHz maximum frequency  Warning: Do not change the initial latency when performing dynamic configuration register reconfiguration.

CR0 Bit	Function	Settings (Binary)
[3]	Fixed Latency Enable	0b: <i>Variable Latency</i> —1 or 2 times Initial latency depending on RWDS during CA cycles. 1b: Fixed 2 times Initial Latency (default). For multi-die stacking only fixed latency is allowed. Variable Latency is not supported by the HyperRAM Controller core IP. This bit must be set to 1 when performing configuration register write.
[2]	Hybrid Burst Enable	0b: Wrapped burst sequences to follow hybrid burst sequencing 1b: Wrapped burst sequences in legacy wrapped burst manner (default)
[1:0]	Burst Length	00b: 128 01b: 64 10b: 16 11b: 32 (default)

Configuration Register 1

Table 16: Configuration Register 1 Bit Assignments

CR0 Bit	Function	Settings (Binary)
[15-12]	Software Reset	1010b: Software Reset Not supported by HyperRAM Controller core.
[11:8]	Reserved	1111b: Reserved (default) When writing this register, these bits should be kept 1111b for future compatibility.
[7]	Reserved	1b: Reserved (default) When writing this register, this bit should be kept 1b.
[6]	Master Clock Type	1b: Single Ended - CK (default) 0b: Differential - CK#, CK
[5]	Hybrid Sleep	1b: Writing 1 to CR1[5] causes the device to enter hybrid sleep (HS) state 0b: Normal operation (default)
[4:2]	Partial Array Refresh	000b: Full array (default) 001b: Bottom 1/2 array 010b: Bottom 1/4 array 011b: Bottom 1/8 array 100b: Reserved 101b: Top 1/2 array 110b: Top 1/4 array 111b: Top 1/8 array
[1:0]	Distributed Refresh Interval	10b: 1 μ s (tCSM) 11b: Reserved 00b: Reserved 01b: 4 μ s (tCSM) Default to 4 μ s by HyperRAM Controller core and cannot be configured.

Interface Designer Settings

When using the HyperRAM Controller with external memories, you need to create GPIO blocks and PLL output clocks in the Efinity® Interface Designer with the settings shown in following tables.

The Ti60 F100S3F2/ Ti35 F100S3F2 FPGA has an embedded HyperRAM memory, so you do not need to create any GPIO to connect to it. Instead, you add a HyperRAM block to your interface design and then connect your RTL design to the block's pins. You also need to use a PLL for calibration as described in [PLL Auto Calibration and PLL Manual Calibration](#) on page 30. Efnix provides an example design targeting this FPGA as described in [HyperRAM Controller Example Design](#) on page 37, which you can use to get started.

PLL Auto Calibration and PLL Manual Calibration

Table 17: Interface Designer Settings for GPIO Resync Mode Output

Block/Bus	Settings					
	Mode	Register Option	Double Data I/O Option	Pull Option	Clock	Inverted Clock
hbc_rst_n	output	register	-	-	-	-
hbc_cs_n	output	register	-	-	hbramClk	-
hbc_ck_p_HI	output	register	resync	-	hbramClk90	Yes
hbc_ck_p_LO	output	register	resync	-	hbramClk90	Yes
hbc_ck_n_HI	output	register	resync	-	hbramClk90	Yes
hbc_ck_n_LO	output	register	resync	-	hbramClk90	Yes
hbc_rwds_OUT_HI [1:0]	output	register	resync	-	hbramClk	Yes
hbc_rwds_OUT_LO [1:0]	output	register	resync	-	hbramClk	Yes
hbc_rwds_OE	output	register	-	-	hbramClk	-
hbc_rwds_IN_HI	input	register	resync	weak pulldown	hbramClk_Cal	Yes
hbc_rwds_IN_LO	input	register	resync	weak pulldown	hbramClk_Cal	Yes
hbc_dq_OUT_HI	output	register	resync	-	hbramClk	Yes
hbc_dq_OUT_LO	output	register	resync	-	hbramClk	Yes
hbc_dq_OE	output	register	-	-	hbramClk	-
hbc_dq_IN_HI	input	register	resync	weak pulldown	hbramClk_Cal	Yes
hbc_dq_IN_LO	input	register	resync	weak pulldown	hbramClk_Cal	Yes

Table 18: Interface Designer Settings for GPIO Normal Mode Output

Block/Bus	Settings					
	Mode	Register Option	Double Data I/O Option	Pull Option	Clock	Inverted Clock
hbc_rst_n	output	none	-	-	-	-
hbc_cs_n	output	register	-	-	hbramClk	-
hbc_ck_p_HI	output	register	normal	-	hbramClk90	Yes
hbc_ck_p_LO	output	register	normal	-	hbramClk90	Yes
hbc_ck_n_HI	output	register	normal	-	hbramClk90	Yes
hbc_ck_n_LO	output	register	normal	-	hbramClk90	Yes
hbc_rwds_OUT_HI [1:0]	output	register	normal	-	hbramClk	-
hbc_rwds_OUT_LO [1:0]	output	register	normal	-	hbramClk	-
hbc_rwds_OE [1:0]	output	register	-	-	hbramClk	-
hbc_rwds_IN_HI [1:0]	input	register	resync	weak pulldown	hbramClk_Cal	-
hbc_rwds_IN_LO [1:0]	input	register	resync	weak pulldown	hbramClk_Cal	-
hbc_dq_OUT_HI [15:0]	output	register	normal	-	hbramClk	Yes
hbc_dq_OUT_LO [15:0]	output	register	normal	-	hbramClk	Yes
hbc_dq_OE [15:0]	output	register	-	-	hbramClk	Yes
hbc_dq_IN_HI [15:0]	input	register	resync	weak pulldown	hbramClk_Cal	-
hbc_dq_IN_LO [15:0]	input	register	resync	weak pulldown	hbramClk_Cal	-

You must ensure that the dynamic phase shift has a 22.5 - 45 degree per step. The formula to calculate a single phase step coverage is given by:

$$\text{Single phase step coverage} = (0.5 \times \text{Post Divider (O)} \times \text{Final Clock Out}) / F_{VCO} \times 360$$



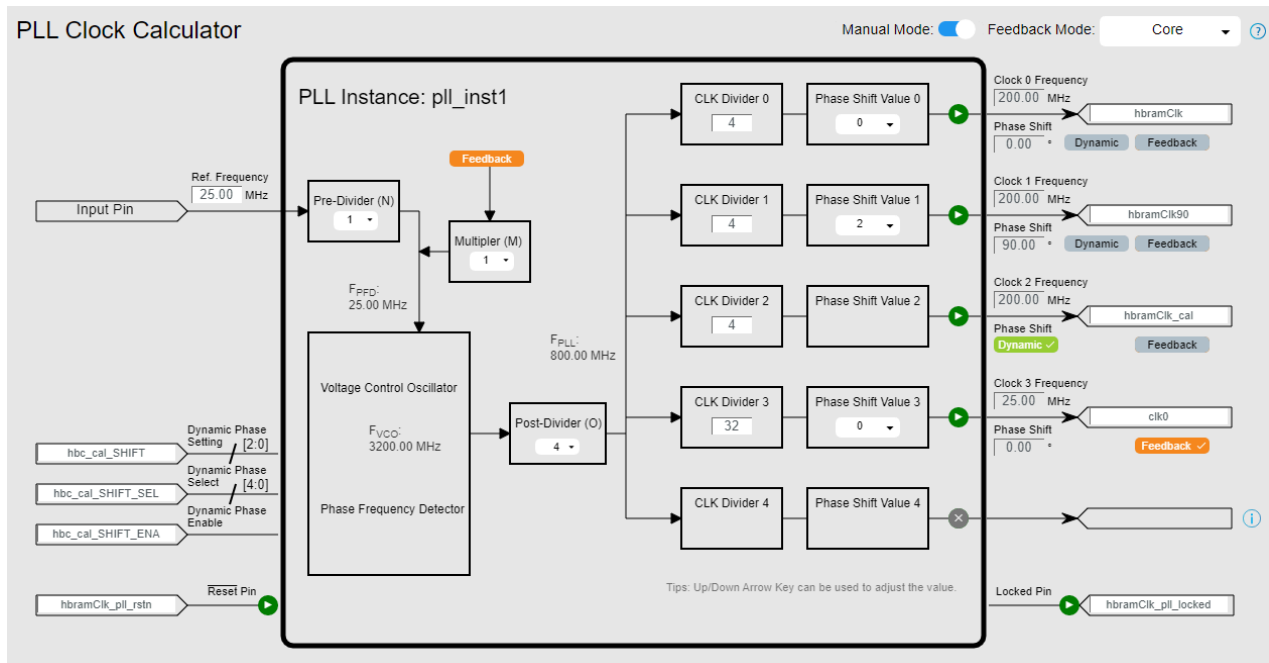
Note: Efinix recommends that you set the **CLK Divider** to 4 to get a 45° per phase shift for the hbramClk, hbramClk90, and hbramClk_cal clocks.

Table 19: PLL Settings for Various Clock Out Frequency

These settings are based on 25 MHz PLL refclk.

Target Clock Out Frequency	F _{VCO}	Post-Divider (O) Setting
250	4000	4
200	3200	4
150	4800	8
125	4000	8
100	3200	8
50	3200	16

Figure 17: PLL Settings Example for 200 MHz Configuration



IP Manager

The Efinity® IP Manager is an interactive wizard that helps you customize and generate Efinix® IP cores. The IP Manager performs validation checks on the parameters you set to ensure that your selections are valid. When you generate the IP core, you can optionally generate an example design targeting an Efinix development board and/or a testbench. This wizard is helpful in situations in which you use several IP cores, multiple instances of an IP core with different parameters, or the same IP core for different projects.



Note: Not all Efinix IP cores include an example design or a testbench.

Generating the HyperRAM Controller Core with the IP Manager

The following steps explain how to customize an IP core with the IP Configuration wizard.

1. Open the IP Catalog.
2. Choose **Memory Controllers > HyperRAM Controller** core and click **Next**. The **IP Configuration** wizard opens.
3. Enter the module name in the **Module Name** box.



Note: You cannot generate the core without a module name.

4. Customize the IP core using the options shown in the wizard. For detailed information on the options, refer to the *Customizing the HyperRAM Controller* section.
5. (Optional) In the **Deliverables** tab, specify whether to generate an IP core example design targeting an Efinix® development board and/or testbench. These options are turned on by default.
6. (Optional) In the **Summary** tab, review your selections.
7. Click **Generate** to generate the IP core and other selected deliverables.
8. In the **Review configuration generation** dialog box, click **Generate**. The Console in the **Summary** tab shows the generation status.



Note: You can disable the **Review configuration generation** dialog box by turning off the **Show Confirmation Box** option in the wizard.

9. When generation finishes, the wizard displays the **Generation Success** dialog box. Click **OK** to close the wizard.

The wizard adds the IP to your project and displays it under **IP** in the Project pane.

Generated Files

The IP Manager generates these files and directories:

- **<module name>_define.vh**—Contains the customized parameters.
- **<module name>_tmpl.v**—Verilog HDL instantiation template.
- **<module name>_tmpl.vhd**—VHDL instantiation template.
- **<module name>.v**—IP source code.
- **settings.json**—Configuration file.
- **<kit name>_devkit**—Has generated RTL, example design, and Efinity® project targeting a specific development board.
- **Testbench**—Contains generated RTL and testbench files.

Customizing the HyperRAM Controller

The core has parameters so you can customize its function. You set the parameters in the **General** tab of the core's IP Configuration window.

Table 20: HyperRAM Controller Core Parameters (Memory Tab)

Parameter	Options	Description
Memory Operating Frequency (MHz)	50, 100, 125, 133, 150, 166, 200, 250	RAM operating frequency in MHz. Default: 200  Note: An operating frequency of more than 200 MHz is only for Ti60F100S3F2/ Ti35F100S3F2 non-L device.
Memory Data Width	8, 16	RAM bit width. Default: 16
Memory Size	32, 64, 128, 256, 512	RAM size in Mb. Default: 256
Calibration Scheme	Manual Calibration, One-off Calibration (PLL Auto Calibration), Auto Periodic Calibration, Periodic Calibration with User Signals, On Demand Calibration (Reset Timer on Calibration), On Demand Calibration (No Reset Timer on Calibration)	Calibration scheme of HyperRAM Controller core. Default: One-off Calibration (PLL auto calibration)
Calibration Days	0 - 511	Timer to perform recalibration in days. E.g.: 7 days 9 hours Default: 7 days
Calibration Hours	0 - 23	Timer to perform recalibration in hours. E.g.: 0 days 6 hours Default: 23 hours
Output Mode	Normal, Resync	Option to select GPIO output mode. Default: Normal
Dual HyperRAM	Enable, Disable	Option to run two HyperRAM at the same time. Only available when PLL Auto Calibration, DDIN Mode is Resync and the correct data width is selected. Default: Disabled
PLL Output Select	PLL Output 0, PLL Output 1, PLL Output 2, PLL Output 3, PLL Output 4	Select the PLL output clock in PLL auto and manual calibration mode. Default: PLL Output 2

Parameter	Options	Description
PLL Output Select 1	PLL Output 0, PLL Output 1, PLL Output 2, PLL Output 3, PLL Output 4	Select the PLL output clock in PLL auto and manual calibration mode for HyperRAM 1. Default: PLL Output 2
PLL Output Select 2	PLL Output 0, PLL Output 1, PLL Output 2, PLL Output 3, PLL Output 4	Select the PLL output clock in PLL auto and manual calibration mode for HyperRAM 2. Default: PLL Output 3
PLL Manual Filter	Enable, Disable	Parameter to filter out incorrectly sampled data. Only applicable in PLL manual calibration flow. Default: Enable
Double Data Rate Input Mode	RESYNC	Indicate the DDIO register mode. Default: RESYNC
User Interface	AXI Native	Select the user interface. Default AXI
AXI3 Data Width	128, 64, 32	AXI data width in bit. Applicable to AXI3 interface. Default: 128
AXI3 AWR Channel Words	16, 32, 64, 128, 256, 512, 1024	AXI AWR channel FIFO depth. Applicable to AXI3 interface. Default: 16
AXI3 R Channel Words	16, 32, 64, 128, 256, 512, 1024	AXI read channel FIFO depth. Applicable to AXI3 interface. Efinix recommends that you to set this parameter to at least double the size of the maximum AXI burst length. For example, if burst length is 256 in the data transaction (io_arw_payload_len[7:0] == 8'h255), set the AXI3 R Channel Words to at least 512. Default: 256
AXI3 W Channel Words	16, 32, 64, 128, 256, 512, 1024	AXI write channel FIFO depth. Applicable to AXI3 interface. Default: 256
Native Data Width	128, 64, 32	Native data width in bit. Applicable to native interface. Default: 32
Write Buffer Depth	16, 32, 64, 128, 256, 512, 1024	Write buffer depth in bit. Applicable to native interface. ⁽²⁾ Default: 256
Read Buffer Depth	16, 32, 64, 128, 256, 512, 1024	Read buffer depth in bit. Applicable to native interface. Default: 256

⁽²⁾ See [Native User Interface](#) on page 16 for more information about setting the buffer depth.

Table 21: HyperRAM Controller Core Parameters (Memory Register Tab)

Parameter	Options	Description
Configuration Register Control	Enable, Disable	Dynamic Reconfiguration of Configuration Register selection. Default: Disable
Initial Latency Count	3 Clocks, 4 Clocks, 5 Clocks, 6 Clocks, 7 Clocks	Define initial latency count. Default: 7 Clocks
Output Drive Strength	19 Ohms, 22 Ohms, 27 Ohms, 34 Ohms, 46 Ohms, 67 Ohms, 115 Ohms	Define output drive strength. Default: 34 Ohms
Hybrid Burst Enable	Hybrid, Legacy	Enable the hybrid burst. Default: Legacy
Wrap Burst Length	16 Bytes, 32 Bytes, 64 Bytes, 128 Bytes	Define wrap burst length. Default: 32 Bytes This setting must be the same as the AXI burst transaction size in wrap mode.
Clock Type	Single-Ended	Define the master clock type. Default: Single-Ended
Partial Array Refresh	None, Full Array, Bottom 1/2 Array, Bottom 1/4 Array, Bottom 1/8 Array, Top 1/2 Array, Top 1/4 Array, Top 1/8 Array	Define the partial array refresh. Default: Full Array

HyperRAM Controller Example Design

You can choose to generate the example design when generating the core in the IP Manager Configuration window. Compile the example design project and download the **.hex** or **.bit** file to your board. To generate example design, the **Example Design Deliverables Option** signal must be enabled.



Important: Efinix tested the example design generated with the default parameter options only.

Efinix provides the following four example designs:

Table 22: HyperRAM Controller Example Designs

All example designs uses the PLL Auto calibration mode. The same design can be used to test out the PLL Manual calibration mode via **Virtual I/O Debugger Settings** on page 39. However, the PLL auto calibration logic is still present in the design.

Example Design	Target	Calibration Mode
Ti60F225_devkit_axi_pll_auto	Titanium Ti60 F225 Development Board	PLL Auto
Ti60F225_devkit_native_pll_auto	Titanium Ti60 F225 Development Board	PLL Auto
Ti60F100_SiP_axi_pll_auto	Ti60 F100S3F2 FPGA	PLL Auto
Ti60F100_SiP_native_pll_auto	Ti60 F100S3F2 FPGA	PLL Auto

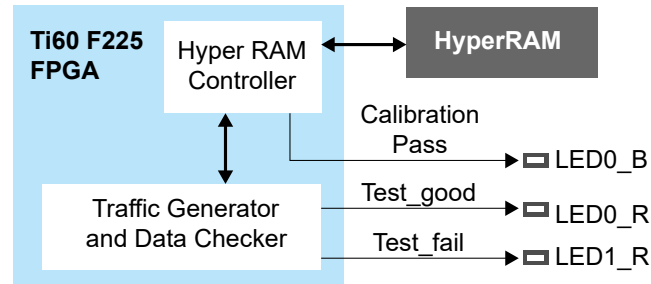
Table 23: Example Design Project Files

File	Description
top.v	Example design top-level wrapper.
ed_encrypt.v	Parameterized of the encrypted HyperRAM Controller file. This file is used by default in the generated example design.
<user_given_ip_name>.v	Generated encrypted HyperRAM Controller file based on user configuration in Efinity IP Manager. Comment-out the EFX_IPM switch in the top.v (line 16) to compile the example design. The example design is meant to target the default IP Manager settings for both AXI and native interface.
efx_ed_hyper_ram_axi_tc.v	Traffic generator and checker for AXI user interface.
efx_ed_hyper_ram_native_tc.v	Traffic generator and checker for native user interface.
efx_crc32.v	CRC32 module used by the traffic generator and checker.
debug_top.v	Verilog file for EFX debug module.
efx_clk_monitor.v	Verilog file for clock estimator module.
debug_profile.json	Virtual I/O Debugger core file. Load this file in the EfinityVirtual I/O Debugger to customize the example design. See Virtual I/O Debugger Settings on page 39.

Titanium Ti60 F225 Development Board

The design performs the memory continuous write and read check at 200 MHz HBRAM Clock. When you run the example design, you should expect the LED0_B to light up indicating that the calibration is passed and LED0_R keeps blinking to indicate that the memory operation is performed correctly. If there is read mismatch, the LED0_R stops blinking while LED1_R lights up to indicate a failed test.

Figure 18: Example Design Block Diagram

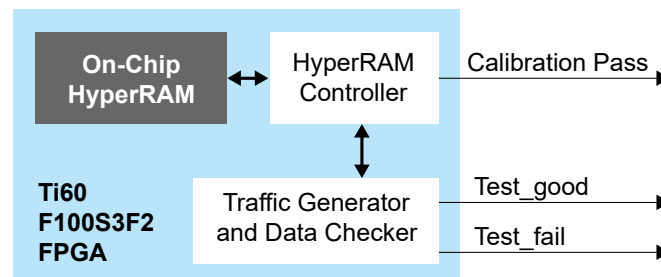


Ti60 F100S3F2 FPGA

This example performs the same function as the one for the Titanium Ti60 F225 Development Board and works in any board using Ti60 F100S3F2 FPGA. The design has the following features:

1. Uses the internal HyperRAM memory inside the package F100S3F2 FPGA.
2. Uses the internal oscillator (intosc) clock to generate the user clock and hyperram clock.
3. You can always monitor the test status via the Efinity® Debugger Virtual I/O feature that is embedded in the example design.

Figure 19: Ti60 F100S3F2 Example Design Block Diagram



Virtual I/O Debugger Settings

The example design includes Efinity Virtual I/O Debugger core for customizing and monitoring the design. The following table describes the Virtual I/O sources and descriptions.



Learn more: Refer to the Debug Perspective: Virtual I/O section of the [Efinity Software User Guide](#) for more information.

Table 24: Virtual I/O Sources and Probes

Name	Width	Radix	Value	Description
s0_Mode	1	Bin	0	Set to 1'b0 to use PLL Auto Calibration mode (Default). Set to 1'b1 to use PLL Manual Calibration mode.
s1_ShiftCtrl_en	1	Bin	0	Set to 1'b1 to enable PLL dynamic phase shift control.
s2_PLL_Shift	3	Bin	000	Total of 8 steps of the PLL dynamic phase shift settings.
p0_ShiftCtrl_en	1	Bin	0	1'b1 indicates that the PLL dynamic phase shift control is enabled.
p1_PLL_Shift	6	Bin	000011	Indicates the following settings: Bit [5:3]: Reserved, default 000. Bit [2:0]: Auto calibrated PLL dynamic phase shift settings
p2_Status	3	Bin	101	Indicates the test status: Bit 2: test_pass (Value toggles when test is passed) Bit 1: test_fail (1'b1 indicates failed test) Bit 0: Calibration pass (1'b1 indicates calibration is passed)
p3_Freq	29	Dec	241850102	Estimated hbram clock frequency. The accuracy is based on the internal oscillator frequency which can be up to $\pm 15\%$ of the desired frequency.
p4_Window_Lock	11	Bin	00001111011	Bit [11:3]: Step Pass Indicator, Bit 0 represent step 0 and up to Bit 7 which represent step 7 Bit [2:0]: Step Locked Value

HyperRAM Controller Testbench

You can choose to generate the testbench when generating the core in the IP Manager Configuration window. To generate testbench, the **Testbench Deliverables Option** signals must be enabled.



Note: You must include all **.v** files generated in the **/Testbench** directory in your simulation.



Important: Efinix tested the testbench generated with the default parameter options only.

Efinix provides a simulation script for you to run the testbench quickly using the Modelsim software. To run the Modelsim testbench script, run `vsim -do modelsim.do` in a terminal application. You must have Modelsim installed on your computer to use this script.

The testbench simulates the example design with fixed latency mode.

Revision History

Table 25: Revision History

Date	Document Version	IP Version	Description
December 2025	4.1	6.12	Corrected signal native_wr_buf_ready description error in HyperRAM Controller Native Ports table. (DOC-2848)
December 2025	4.0	6.12	Corrected native_rd_valid in table HyperRAM Controller Native Ports. (DOC-2823) Corrected statement in PLL Auto Calibration Flow and flow information in Calibration Flow Diagram. Corrected Calibration Scheme, Write Buffer Depth, and Read Buffer Depth in table HyperRAM Controller Core Parameters (Memory Tab).
November 2025	3.9	6.12	Updated double-data rates in Features. (DOC-2689) Updated Interface Designer Settings for PLL Auto Calibration and PLL Manual Calibration. Added Memory Operating Frequency of 250 MHz and added a note in Customizing the HyperRAM Controller. Added dynamic reconfiguration and hybrid sleep in Features. (DOC-2745) Added new topic On-Demand Calibration, Dynamic Reconfiguration for Configuration Register, and Configuration Register. In Customizing the HyperRAM Controller, added Calibration Scheme and removed Calibration Mode, Periodic Calibration, and Periodic Calibration Signal in table HyperRAM Controller Core Parameters (Memory Tab). Added Configuration Register Control in table HyperRAM Controller Core Parameters (Memory Register Tab) Added table Periodic Calibration User Ports in Ports.
August 2025	3.8	6.11	Added note to refer to Ti60/Ti35 data sheets for t _{CSM} specifications under native_ram_burst_len [11:0] in HyperRAM Controller Native Ports. (DOC-2536) Updated values for io_arw_payload_burst [1:0] in HyperRAM Controller AXI Ports. Added additional descriptions for native_ram_address. (DOC-2648) Updated boundary values in table 512 Mb HyperRAM Native Address Boundary.
February 2025	3.7	6.9	Removed note from the Memory Operating Frequency in the Customizing the HyperRAM Controller. (DOC-2398)
January 2025	3.6	6.8	Updated the PLL Manual Calibration Flow chapter and added PLL Manual Filter in Customizing the HyperRAM Controller section. (DOC-2315)
December 2024	3.5	6.7	Updated table HyperRAM Controller Core Parameters (Memory Tab). (DOC-2269)
November 2024	3.4	6.5	Updated IP Version in Revision History. (DOC-2185)

Date	Document Version	IP Version	Description
November 2024	3.3	6.5	Added Topaz in Device Support. (DOC-2176) Added IP Version in Revision History. (DOC-2185)
October 2024	3.2	-	Updated Table 4: HyperRAM Controller AXI Ports on page 10 and added a paragraph in AXI Addressing on page 14. (DOC-2013) Updated Table 20: HyperRAM Controller Core Parameters (Memory Tab) on page 34, Table 18: Interface Designer Settings for GPIO Normal Mode Output on page 31, Introduction, Features, topic AXI Addressing, and Native User Interface. (DOC-2097) Added topic Periodic Recalibration Flow, Table 8: 512 Mb HyperRAM AXI Address Boundary on page 14, and Table 9: 512 Mb HyperRAM Native Address Boundary on page 16. Removed soft logic calibration block diagram and Soft Logic Calibration topic. Removed Trion. Added Topaz in Device Support and document.
May 2024	3.1	-	Added topic AXI Interface. (DOC-1875) Added feature Dual HyperRAM support. Added figure HyperRAM Controller Dual HyperRAM Block Diagram. Updated table HyperRAM Controller Ports with second HyperRAM ports, AXI Address Conversion Ratio, and HyperRAM Controller Core Parameters (Memory Tab) for Dual HyperRAM. Updated figure HyperRAM Controller PLL Calibration Block Diagram and HyperRAM Controller PLL Manual Calibration Block Diagram with <code>hbc_cal_debug_info[26:0]</code> .
April 2024	3.0	-	Removed Data and RWDS calibration in Features. (DOC-1827) Updated Resource Utilization and Performance. Updated figures HyperRAM Controller PLL Calibration Block Diagram, HyperRAM Controller PLL Manual Calibration Block Diagram, and HyperRAM Controller Soft Logic Calibration Block Diagram (change to <code>hbc_cal_debug_info[23:0]</code>). Added Bit [23:16] are reserved in table HyperRAM Controller Ports (<code>hbc_cal_debug_info[15:0]</code>), HyperRAM Controller Core Parameters (Memory Tab), and Virtual I/O Sources and Probes. Updated topic PLL Auto Calibration Flow and figure Calibration Flow Diagram, topic PLL Manual Calibration Flow. Changed sub-topic AXI Interface to Hyperbus Interface, figure name of AXI Interface Write Operation Waveform and AXI Interface Read Operation Waveform to Hyperbus Write Operation Waveform and Hyperbus Read Operation Waveform. (DOC-1825)

Date	Document Version	IP Version	Description
March 2024	2.9	-	Updated table HyperRAM Controller Native Ports. (DOC-1749) Added new paragraph in Native User Interface topic. Updated figure Native Interface Write Operation Waveform.
December 2023	2.8	-	Removed Hybrid Sleep Enable parameter. (DOC-1600)
November 2023	2.7	-	Updated supported data rate. (DOC-1535)
April 2023	2.6	-	Added note about the HyperRAM verified in hardware. (DOC-1216) Corrected hbc_rwds_OUT_HI and hbc_rwds_OUT_LO signal widths. (DOC-1236)
March 2023	2.5	-	Updated description for Wrap Burst Length parameter. (DOC-1187)
February 2023	2.4	-	Added note about the resource and performance values in the resource and utilization table are for guidance only. Corrected native_wr_data and native_rd_data ports widths.
September 2022	2.3	-	Added note about only one HyperRAM Controller can be instantiated for a single device. (DOC-909) Updated native port descriptions, added FIFO depth size calculation, and added native mode operation waveform. (DOC-846)
March 2022	2.2	-	Corrected supported bit width to 16x and maximum double data-rate to 1000 Mbps. (DOC-748)
January 2022	2.1	-	Added read and write operation waveforms. Updated resource utilization table. (DOC-700)
December 2021	2.0	-	Added support for PLL manual calibration mode. Added support for native user interface. Updated parameters supported in IP Manager. Updated HyperRAM Controller ports. Updated example design.
October 2021	1.1	-	Added note to state that the f_{MAX} in Resource Utilization and Performance, and Example Design Implementation tables were based on default parameter settings. Updated design example target board to production Titanium Ti60 F225 Development Board and updated Resource Utilization and Performance, and Example Design Implementation tables. (DOC-553)
June 2021	1.0	-	Initial release.